Tokens, Lexemes, and Patterns

Lexical Analysis

Presented by: _____

Introduction

- Tokens, lexemes, and patterns are important concepts in lexical analysis.
- Lexical analysis is the first phase of a compiler.
- It converts source code into meaningful units.

What is a Token?

- A token is a meaningful unit identified by the lexical analyzer.
- Common types: Keywords, Identifiers, Operators, Constants, Special symbols.

Token Example

For the statement: `int age = 20;`

`int` → Keyword

`age` → Identifier

`=` → Operator

`20` → Constant

`;` → Special symbol

What is a Lexeme?

- A lexeme is the actual sequence of characters in the source program.
- It matches a particular token.

Example: `int age = 20;`

- `int` is a lexeme of the keyword token.
- `age` is a lexeme of the identifier token.
- `20` is a lexeme of the constant token.

What is a Pattern?

- A pattern is a rule used to describe a token.
- It explains how a particular token can be formed.
- Patterns are often represented using regular expressions.

`Identifier` $\rightarrow$ `letter(letter | digit)*`

Difference Between Them

Term	Meaning	Example
Token	Category of a word or symbol	Identifier
Lexeme	Actual text in the program	count
Pattern	Rule for recognizing it	Letter followed by letters or digits

Complete Example

For the statement: `count = 10;`

- `count` → Lexeme
- Identifier → Token
- `letter(letter | digit)*` → Pattern
- = → Operator token
- `10` → Constant token

Easy Way to Remember

- Token = Category
- Lexeme = Actual text
- Pattern = Rule for recognizing it

Conclusion

- Tokens, lexemes, and patterns help the lexical analyzer understand source code.
- They organize the program into meaningful units.
- The resulting tokens are passed to the next phase of the compiler.

Thank You